



INSTITUTO POLITÉCNICO NACIONAL

UPIICSA

ABSTRACCIÓN Y USO DE DATOS

EQUIPO: SYNTAX ERROR

3CM32

REPORTE PRÁCTICA 20

Desarrollo: Merge sort de manera indirecta para ordenar conjunto de caracteres, enteros y nuevo tipo de dato (PE y POO), reorganizando índices o punteros sin modificar los datos originales.

Errores:

- Error 1: Modificar o desplazar las estructuras originales en memoria en lugar de trabajar exclusivamente con un arreglo auxiliar de índices o punteros.
- Error 2: Fugas de memoria por no liberar los arreglos auxiliares temporales creados durante la fase de mezcla (*merge*).

Soluciones:

- Solución 1: Definir la colección a ordenar como un arreglo de punteros y hacer las reasignaciones únicamente sobre dichas referencias.
- Solución 2: Asegurar la llamada explícita a delete[] de los arreglos temporales al concluir cada proceso de mezcla.

Código:

```cpp
#include <iostream>
#include <vector>
#include <string>
#include <functional>
#include <limits>

using namespace std;

template <typename T>
class MergeSort {
private:
    vector<T> datos;
    function<bool(const T&, const T&)> comparar;

    void mezclar(int izq, int medio, int der) {
        vector<T> izquierda(datos.begin() + izq, datos.begin() + medio + 1);
        vector<T> derecha(datos.begin() + medio + 1, datos.begin() + der + 1);

        size_t i = 0, j = 0;
        int k = izq;
```

```
        while (i < izquierda.size() && j < derecha.size()) {
            if (!comparar(derecha[j], izquierda[i]))
                datos[k++] = izquierda[i++];
            else
                datos[k++] = derecha[j++];
        }
        while (i < izquierda.size()) datos[k++] = izquierda[i++];
        while (j < derecha.size())   datos[k++] = derecha[j++];
    }

    void ordenar(int izq, int der) {
        if (izq >= der) return;
        int medio = izq + (der - izq) / 2;
        ordenar(izq, medio);
        ordenar(medio + 1, der);
        mezclar(izq, medio, der);
    }

public:
    MergeSort(const vector<T>& v,
              function<bool(const T&, const T&)> criterio =
                  [](const T& a, const T& b) { return a < b; })
        : datos(v), comparar(criterio) {}

    void ordenar() {
        if (!datos.empty())
            ordenar(0, static_cast<int>(datos.size()) - 1);
    }

    const vector<T>& obtener() const { return datos; }

    void mostrar(const string& titulo) const {
        cout << titulo << ": ";
        for (const T& x : datos) cout << x << "  ";
        cout << "\n";
    }
};

class Estudiante {
private:
    string nombre;
    int    edad;
    double promedio;

public:
    Estudiante(string n = "", int e = 0, double p = 0.0)
        : nombre(n), edad(e), promedio(p) {}
```

```
    string getNombre()   const { return nombre; }
    int    getEdad()     const { return edad; }
    double getPromedio() const { return promedio; }

    bool operator<(const Estudiante& otro) const {
        return nombre < otro.nombre;
    }

    friend ostream& operator<<(ostream& os, const Estudiante& e) {
        os << "[" << e.nombre << ", " << e.edad << " años, prom: "
           << e.promedio << "]";
        return os;
    }
};

template <typename T>
T leerValor(const string& mensaje) {
    T valor;
    while (true) {
        cout << mensaje;
        if (cin >> valor) return valor;
        cout << "Entrada no válida, intente de nuevo.\n";
        cin.clear();
        cin.ignore(numeric_limits<streamsize>::max(), '\n');
    }
}

int leerCantidad() {
    int n;
    do {
        n = leerValor<int>("Cantidad de elementos: ");
        if (n <= 0) cout << "Debe ser mayor que cero.\n";
    } while (n <= 0);
    return n;
}

string leerTexto(const string& mensaje) {
    string s;
    while (true) {
        cout << mensaje;
        cin >> ws;
        getline(cin, s);
        if (!s.empty()) return s;
    }
}

void ordenarEnteros() {
    int n = leerCantidad();
```

```
    vector<int> v;
    for (int i = 0; i < n; i++)
        v.push_back(leerValor<int>("Entero " + to_string(i + 1) + ": "));

    MergeSort<int> ms(v);
    ms.ordenar();
    ms.mostrar("\nEnteros ordenados");
}

void ordenarCaracteres() {
    int n = leerCantidad();
    vector<char> v;
    for (int i = 0; i < n; i++)
        v.push_back(leerValor<char>("Carácter " + to_string(i + 1) + ": "));

    MergeSort<char> ms(v);
    ms.ordenar();
    ms.mostrar("\nCaracteres ordenados");
}

void ordenarEstudiantes() {
    int n = leerCantidad();
    vector<Estudiante> v;
    for (int i = 0; i < n; i++) {
        cout << "\nEstudiante " << i + 1 << "\n";
        string nombre = leerTexto("Nombre: ");
        int edad = leerValor<int>("Edad: ");
        double promedio = leerValor<double>("Promedio: ");
        v.push_back(Estudiante(nombre, edad, promedio));
    }

    int opcion;
    do {
        opcion = leerValor<int>(
            "\nOrdenar por:\n1. Nombre\n2. Edad\n3. Promedio (mayor a
menor)\nOpción: ");
    } while (opcion < 1 || opcion > 3);

    function<bool(const Estudiante&, const Estudiante&)> criterio;
    if (opcion == 1)
        criterio = [](const Estudiante& a, const Estudiante& b) { return a < b;
};
    else if (opcion == 2)
        criterio = [](const Estudiante& a, const Estudiante& b) { return
a.getEdad() < b.getEdad(); };
    else
        criterio = [](const Estudiante& a, const Estudiante& b) { return
a.getPromedio() > b.getPromedio(); };
```

```
    MergeSort<Estudiante> ms(v, criterio);
    ms.ordenar();

    cout << "\nEstudiantes ordenados:\n";
    for (const auto& e : ms.obtener()) cout << "  " << e << "\n";
}

int main() {
    int opcion;
    do {
        cout << "\nMerge Sort\n"
             << "1. Ordenar enteros\n"
             << "2. Ordenar caracteres\n"
             << "3. Ordenar estudiantes\n"
             << "0. Salir\n";
        opcion = leerValor<int>("Opción: ");

        switch (opcion) {
            case 1: ordenarEnteros();     break;
            case 2: ordenarCaracteres();  break;
            case 3: ordenarEstudiantes(); break;
            case 0: cout << "Hasta luego.\n"; break;
            default: cout << "Opción no válida.\n";
        }
    } while (opcion != 0);

    return 0;
}
```